

Project Report & Interview Preparation

Transformer Attention Accelerator

Full RTL Implementation in SystemVerilog — Verified via Simulation

✓ End-to-End Correctness Target Met

0.00527

Max Absolute Error
(target < 0.01)

~13,200

Cycles per
Attention Pass

8

Testbenches,
All Passing

4

Root-Caused &
Fixed Bugs

Generated from verified simulation results · SystemVerilog / Vivado xsim · NumPy golden-reference verification

Part 1 — Project Report

Architecture, measured results, precision engineering, and known scope boundaries

1. Architecture Overview

A synthesizable, four-stage attention pipeline computing $O = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$ entirely in fixed-point RTL, with sequence length and head dimension $N = D = 64$, using Q8.8 fixed-point (16-bit: 8 integer + 8 fractional bits) for all external interfaces.

Stage	Module	Role
① Score	qk_systolic.sv	Systolic PE array computing raw $Q \cdot K^T$ (output-stationary, skewed feed)
② Scale + Normalize	softmax_unit.sv	Row-sequential FSM: divides by $\sqrt{d_k}$, finds row max, looks up $\exp()$ via LUT, computes $1/\text{row_sum}$ via Newton-Raphson, normalizes
③ Blend	av_multiply.sv	Systolic PE array computing $A \cdot V$ (column-fed V, no transpose trick available)
Reciprocal	reciprocal_nr.sv	Standalone 1/D approximation: bit-shift seed + 3 Newton-Raphson iterations, no hardware divider
Control	top_fsm.sv	Master sequencer: drives start/done handshakes and generates the skewed feeds for both systolic stages from full-array data
Memory front-end	bram_if.sv	Serial load of Q/K/V (one word/cycle) → full-array handoff; full-array O capture → serial drain out. Built and verified standalone; not yet wired into <code>top_fsm</code> .

Key design decisions

- **Q8.8 externally, wider internally where needed.** The `exp()` LUT and running sum use 16 fractional bits internally; attention weights are emitted as Q1.15 (not Q8.8) since Q8.8's 1/256 resolution was too coarse for probability-valued weights.
- **No hardware divider.** `1/row_sum` is computed via a seed (priority-encoder bucket + 8-entry mantissa-refined lookup table, computed at elaboration time) followed by 3 Newton-Raphson iterations.
- **Scope: simulation-only.** No Vivado synthesis/implementation was run for this phase — see §5 for what that means for the numbers below.

2. Final Correctness Result

The definitive end-to-end test (`tb_top_fsm.sv`) feeds real Q/K/V test vectors through the complete RTL pipeline and compares the dequantized output against an independent NumPy float32 reference (`scripts/gen_golden.py`), computing the exact metric the project's correctness bar is defined in.

Metric	Measured	Target	Result	Margin
Max Absolute Error	0.005270	< 0.01	PASS	~1.9×
Mean Absolute Error	0.001982	< 0.005	PASS	~2.5×
Elements exceeding tolerance	0 / 4096	0	PASS	—

What this actually certifies: the real SystemVerilog RTL, simulated in Vivado xsim, processing the same Q/K/V data (quantized to Q8.8) that was fed to a from-scratch NumPy implementation of the true attention formula, produces output within 0.53% max absolute error of the floating-point reference — a direct, apples-to-apples correctness check, not a self-consistency check against the RTL's own arithmetic.

3. Performance / Latency

Stage	Cycles (measured/derived)	% of total
QK Score (<code>qk_systolic</code>)	~194	1.5%
Scale + Normalize (<code>softmax_unit</code>)	~12,736	96.5%
Blend (<code>av_multiply</code>)	~194	1.5%
Total, one full attention pass	~13,200 cycles	100%

Measured from `tb_top_fsm` 's simulation timestamp (132,015 ns at the 10 ns/cycle testbench clock period $\approx$ 13,200 cycles). This is behavioral-simulation cycle count, not post-synthesis timed performance (no clock frequency/MHz figure exists — see §5).

Note on latency: softmax dominates at ~96% of total runtime because it's a deliberately simple, non-pipelined-across-rows design (each of the 64 rows is processed in three sequential passes: find max, compute exp+sum, normalize). Overlapping rows (e.g. starting row $i+1$'s max-scan during row i 's normalize pass) would cut this significantly but was scoped out in favor of first getting a correct, verifiable design.

4. Precision Engineering: Bugs Found, Root-Caused, and Fixed

Four distinct issues were found through simulation and testbench work, each root-caused to a specific mechanism and verified fixed with before/after measurements. This is arguably the most substantive engineering content of the project — see Part 2 for interview framing of each.

Bug 1 — Newton-Raphson Reciprocal Seed Collapse

Module: `reciprocal_nr.sv` | Found via: hand-verification of an external code-review claim

The reciprocal's initial seed (used to bootstrap Newton-Raphson) was accurate only to within $2\times$ of the true value in the worst case. When the seed landed close to exactly $2\times$ off, the NR correction factor $(2 - D \cdot r)$ collapsed to ≈ 0 , destroying the estimate instead of refining it — and Q8.8's coarse resolution meant it could never recover in the remaining iterations.

BEFORE

D=511 $\rightarrow$ r=1

(true value: 128 — $\sim 50\%$ error, non-recoverable)

AFTER

D=511 $\rightarrow$ r=128

(exact, converges in 1 NR iteration)

Fix: replaced the coarse (bucket-only) seed with an 8-entry table keyed on 3 additional mantissa bits, narrowing worst-case seed error from $\sim 2\times$ to $\sim 6\%$. **Verified:** exhaustive sweep of all 65,280 valid inputs ($D = 256\text{--}65,535$), 0 failures.

Bug 2 — exp() LUT Precision Too Coarse for Small Values

Module: softmax_unit.sv / scripts/gen_exp_lut.py | Found via: directed "peaked distribution" testbench case

The exp() lookup table stored values at Q8.8 precision (1/256 resolution). For a token scoring well below a row's max, the true exp() value (e.g. ~0.0025) was smaller than the table's own resolution, forcing it to round up to the nearest representable step — a ~57% relative overshoot on that single entry. With up to 63 such entries summed per row, the overshoots compounded into a measurable, systematic bias.

BEFORE

weight = 0.803

(true: 0.865 — largest single-weight error found)

AFTER

weight = 0.868

(within ~0.3% of true)

Fix: widened the LUT and running-sum accumulator to 16 internal fractional bits (double Q8.8), truncating back to Q8.8 only once at the very end instead of once per LUT entry before summing.

Bug 3 — Attention-Weight / Reciprocal Output Precision Floor

Modules: softmax_unit.sv, reciprocal_nr.sv, av_multiply.sv | Found via: Python-modeled precision sweep before committing to RTL changes

Even with Bug 2 fixed, storing the final attention weights and the reciprocal result at Q8.8 precision capped achievable end-to-end accuracy at ~0.07 max error — probabilities are always <1.0, so Q8.8 wastes its integer range and leaves only 1/256 resolution for values that are often near 0.01. A faithful Python model of the fixed-point pipeline showed that widening *only* the weights (to Q1.15) still left max error at 0.0124 — still failing; *both* the weights and the reciprocal needed widening together.

BEFORE (Q8.8 BOTH)

max err 0.091

AFTER (Q1.15 + 16-BIT RECIP.)

max err 0.062*

*This intermediate number still failed — it was actually Bug 4, hiding underneath this fix, that finally closed the gap. See below.

Fix: attention weights re-formatted to Q1.15 (clamped to 0x7FFF to stay signed-positive for the downstream multiplier); reciprocal output widened from 8 to 16 fractional bits. Required modifying `softmax_unit`, `reciprocal_nr`, and `av_multiply`'s output truncation together.

Bug 4 — Double-Rounding in a Scaling Constant (int'() vs \$rtoi())

Module: softmax_unit.sv | Found via: isolate-then-trace methodology (see Part 2, Q6)

The `1/√64` scaling constant was computed as `int'(0.125 × 256 + 0.5)`. `0.125 × 256 = 32.0` exactly — no rounding needed. But SystemVerilog's `int'()` cast already rounds to nearest on its own (unlike `$rtoi()`, which truncates and is what the "+0.5" idiom is correct for). Adding 0.5 to an already-exact value created an artificial tie at 32.5, which rounded *up* to 33 — a silent, systematic **3.125% scaling error applied to every delta value computed in softmax**, in every row.

BEFORE

max err 0.0617

(1533/4096 elements failing)

AFTER

max err 0.00527

(0/4096 elements failing — final PASS)

Fix: removed the redundant `+ 0.5`. **Verified:** a purpose-built diagnostic testbench (`tb_softmax_debug.sv`) traced the exact internal signal (a running-sum accumulator) that first diverged from its hand-derived expected value, confirming the fix at the source before re-running the full pipeline.

Related methodology finding (not an RTL bug)

Unrealistic test-data range caused Q8.8 overflow. Early end-to-end runs showed a catastrophic max error of 6.55 — traced to golden test vectors generated with too large an input range ($Q/K \sim \pm 4.0$), causing the score matrix *S* to reach ± 175 against Q8.8's ± 128 representable range. Regenerating golden data with a realistic range (± 2.0 , consistent with how real attention keeps $Q/K \sim O(1)$) dropped max error to 0.091 immediately, with zero RTL changes — confirming the RTL itself was already correct and the issue was test methodology, not the design.

5. Test Coverage

Testbench	Method	Result
<code>tb_qk_systolic.sv</code>	Internal bit-exact golden model (5 layers: zeros, identity, hand-checked, random ×15, back-to-back)	PASS
<code>tb_qk_systolic_golden.sv</code>	Independent cross-check vs. NumPy-computed fixed-point reference, signed inputs	PASS — 4096/4096 exact match
<code>tb_av_multiply.sv</code>	Internal bit-exact golden model (5 layers incl. identity-blend structural check)	PASS

<code>tb_reciprocal_nr.sv</code>	Exhaustive sweep, all 65,280 valid inputs vs. float reference	PASS
<code>tb_softmax_unit.sv</code>	Tolerance-based vs. true floating-point softmax (4 layers incl. peaked/low-entropy case)	PASS
<code>tb_top_fsm.sv</code>	End-to-end vs. independent NumPy float32 golden reference	PASS — 0.00527 max err
<code>tb_bram_if.sv</code>	Data-integrity: sequential + random load/drain, back-to-back round trips	PASS — 12/12

Plus 2 targeted diagnostic testbenches built specifically during debugging (not part of the regression suite):

`tb_reciprocal_wide.sv` (isolated the wide-precision reciprocal config) and `tb_softmax_debug.sv` (internal-signal tracing that found Bug 4).

6. Known Limitations & Explicit Scope Boundaries

No synthesis was run. There are no real LUT/DSP/BRAM/FF utilization numbers, no achievable clock frequency, and no timing-closure (WNS) data. Every number in this report comes from behavioral simulation. This was a deliberate scope decision (see Part 2, Q9) — not an oversight.

- **DSP budget (architectural, unresolved):** `qk_systolic` 's current design is a one-shot 64×64 spatial systolic array — 4,096 PEs. The KV260 target board has 1,728 DSP slices. This design would not fit as-is; a tiled/reused smaller array (e.g. 16×16, reused across passes) would be needed before real synthesis. Flagged early, never resolved, since synthesis was out of scope.
- **No hardware deployment.** Verified entirely via SystemVerilog simulation + Python golden-model cross-checks, not on physical FPGA hardware.
- `bram_if.sv` **built and verified standalone, not yet integrated.** No top-level wrapper connects it to `top_fsm` yet — the two are separately-tested modules.
- **No ping-pong buffering** in `bram_if` — strictly sequential load → compute → drain, no overlap across passes.
- **Single golden dataset** used for the final end-to-end PASS — one random seed, 4096 output elements. Broader confidence would come from testing multiple independent random datasets.

Part 2 — Interview Question Packet

Likely questions, organized by category, with ready answers grounded in what was actually built and measured

A. Architecture & Design

1 Walk me through your attention accelerator architecture.

Four stages: a systolic array computes raw $Q \cdot K^T$; a row-sequential FSM scales by $1/Vd_k$, computes softmax via an $\exp()$ LUT and a Newton-Raphson reciprocal (no hardware divider); a second systolic array blends the attention weights with V ; and a master FSM sequences all three stages, generating the skewed data feeds each systolic stage needs. Everything communicates in Q8.8 fixed-point externally, with wider internal precision at two specific points where Q8.8 wasn't accurate enough.

2 Why did you use a systolic array for the matrix multiplies?

Regular, local data flow with high data reuse — each PE only talks to its neighbors, no broadcast network, which maps efficiently onto FPGA fabric. For $Q \cdot K^T$ specifically, feeding K row-wise into columns produces the transpose "for free," with zero actual transpose hardware — a row of K is a column of K^T by definition, so no data reordering is needed.

3 Q and K get the same row-wise feed, but you told me V is fed differently for the AV stage. Why?

$S = Q \cdot K^T$ is special: both operands contract over their own trailing dimension, so both can be fed row-wise with no transpose trick needed. $O = A \cdot V$ is a plain matmul — no such symmetry. A contracts over its column index (row-fed, like Q), but V contracts over its *row* index, so V has to be streamed by column instead. I derived this from first principles using a "which index is time-varying vs. which is fixed per PE" argument, not by assuming the two stages would look alike.

4 Why fixed-point instead of floating-point?

Floating-point needs much larger IP blocks (wide multipliers, exponent alignment/normalization logic) — expensive in LUTs/DSPs and slower to clock. Fixed-point uses fewer resources and higher achievable frequency, and for attention specifically, careful bit-width selection gets you well within acceptable error — I measured 0.00527 max absolute error against a float32 reference, comfortably under a 0.01 target.

B. Verification & Debugging

5 How did you verify correctness?

Two independent methods layered together. Per-module: internal bit-exact golden models (mirroring the RTL's own truncation/rounding logic, catching wiring bugs fast without external dependencies) plus, for the trickiest modules, an independent NumPy-computed cross-check that couldn't share the RTL's own blind spots. End-to-end: the real RTL, in simulation, fed the same data as a from-scratch NumPy implementation of the true attention formula, compared directly in real-valued terms against the doc-specified 0.01 max-error bar — not a proxy metric.

6 Tell me about a bug you found and how you debugged it.

After a first precision fix, my end-to-end error dropped but plateaued about 6× above target — and didn't match what my own reference model predicted, telling me a second bug was still hiding. Instead of continuing to guess, I isolated the problem: I first exhaustively verified my reciprocal submodule against a floating-point reference across its entire valid input range, in isolation, and confirmed it was mathematically exact. That ruled it out and pointed at how the parent module was using it. I then built a targeted diagnostic testbench that printed the parent's internal signals directly, for one case I'd already hand-computed expected values for. The very first internal signal — an accumulator — was off by about 2%. Tracing backward led to a scaling constant that should have been exactly 32 but was computing to 33, from a double-rounding bug: I'd added a "+0.5 before casting" idiom that's correct for one SystemVerilog function but wrong for the cast operator I'd actually used, which already rounds on its own. Fixing that closed the gap and the full pipeline passed with real margin.

This is the strongest debugging story from the project — isolate-the-variable methodology, not guesswork, and a satisfying root cause with a clear "why."

7 What edge cases did your testbenches cover?

All-zero inputs (structural sanity check), identity-matrix blends (bit-exact expected result with no truncation error, a strong test of skew-feed timing specifically), an exhaustive sweep of the reciprocal's entire valid input range (not just samples — this is exactly what caught a bug that only showed up at specific values near power-of-two boundaries), directed "peaked distribution" cases for softmax (low-entropy rows stress LUT precision differently than random data), constrained-random stress testing, and back-to-back runs with no reset between them to verify state doesn't leak across passes.

8 Did you run any formal verification?

No — directed and random testbenches with independent golden-reference comparison, plus one exhaustive sweep where the input space was small enough to make that fully tractable (65,280 values for the reciprocal). For a full tapeout, I'd add formal property checking, particularly around the FSM handshake timing between stages.

C. Fixed-Point / Precision Engineering

9 You used Q8.8 in most places but widened precision elsewhere. Walk me through that decision.

Q8.8 is fine when values span a wide dynamic range including large integers — but attention weights are always in $[0,1]$, so Q8.8 wastes its 8 integer bits on a range that's never used, leaving only $1/256$ resolution for numbers that are often near 0.01. I found this empirically, not by assumption: I built a faithful Python model of the exact fixed-point pipeline and swept precision configurations before touching RTL, which showed widening only one of (attention weights, reciprocal) still failed — both needed to move to higher fractional-bit formats together to close the gap. That measure-before-cutting approach saved me from doing partial surgery that wouldn't have worked.

10 How does your softmax handle numerical overflow / stability?

Standard max-subtraction trick: subtract the row's max score before exponentiating, so every input to `exp()` is ≤ 0 and the result is bounded in $(0,1]$ — mathematically identical to the naive formula since it's a shift-invariance property of softmax, but avoids exponentiating a raw score that could otherwise overflow. Separately, I found and fixed a real overflow bug: my *test data* was originally generated with too wide an input range, causing the raw QK^T scores themselves (before softmax even runs) to exceed the ± 128 range Q8.8 can represent, wrapping silently. That was a test-methodology bug, not a design flaw — regenerating test data at a realistic range fixed it with zero RTL changes.

11 You avoided a hardware divider for the softmax reciprocal. How does that work, and what went wrong the first time?

Newton-Raphson: seed a rough estimate from the input's bit position (no multiplier needed), then refine with $r \leftarrow r \cdot (2 - D \cdot r)$, which roughly doubles correct bits per iteration. The bug: my original seed used only the coarse power-of-two bucket, giving up to $2\times$ worst-case error — and I hadn't realized that when the seed is *exactly* that bad, the correction term collapses to zero on the first iteration instead of refining, permanently destroying the estimate rather than degrading it gracefully. I fixed it with a finer 8-entry seed table ($\sim 6\%$ worst-case error) and verified the fix with an exhaustive sweep, not just the specific failing case, since the failure mode only appears at specific input values.

D. Tradeoffs & What's Next

12 What's left before this could actually run on hardware?

Two things, honestly: first, resolving a known DSP-budget mismatch — my current QK systolic array is a one-shot 64×64 grid (4,096 PEs), and the target board only has 1,728 DSP slices, so it wouldn't fit as synthesized today; it would need a tiled, reused smaller array. Second, actually running synthesis and implementation to get real timing/utilization numbers, which I deliberately scoped out of this phase to focus on getting correctness fully verified first. I have a memory front-end module (`bram_if`) built and independently verified, but not yet wired into the top-level design.

13 Why didn't you just chase resource optimization or synthesis from the start?

Sequencing risk deliberately: correctness bugs are much cheaper to find and fix in behavioral simulation, where I can directly observe internal signals and get instant golden-model comparisons, than after synthesis where debugging is slower and noisier. I wanted a fully verified, trustworthy design before spending effort on an architecture change (tiling the PE array) that only matters once I'm confident the underlying math is right. That ordering paid off directly — I found and fixed four real precision bugs during this phase that would have been much more painful to isolate post-synthesis.

14 How would you reduce latency if you needed to?

Softmax is ~96% of total latency because it processes 64 rows fully sequentially, three passes each. The obvious win is pipelining across rows — start row $i+1$'s max-scan while row i 's normalize pass is still finishing — which the current design deliberately doesn't do, favoring simplicity for the first correct version. Beyond that: more parallel `exp()`-LUT read ports, or restructuring the three-pass-per-row structure to overlap more aggressively.

15 What would you do differently if you started over?

I'd build the precision-modeling Python harness (a faithful bit-exact mirror of the fixed-point pipeline) earlier — it's what let me test precision changes and predict their impact before touching RTL, and by the time I built it partway through, it immediately paid for itself by catching that a partial fix (widening only the weights) wouldn't have been sufficient, before I wasted a synthesis-and-test cycle finding that out the hard way.